BULKOR: Enabling Bulk Loading for Path ORAM

Xiang Li[1], Yunqian Luo[1], Mingyu Gao[1,2]

1 Tsinghua University

2 Shanghai Qi Zhi Institute

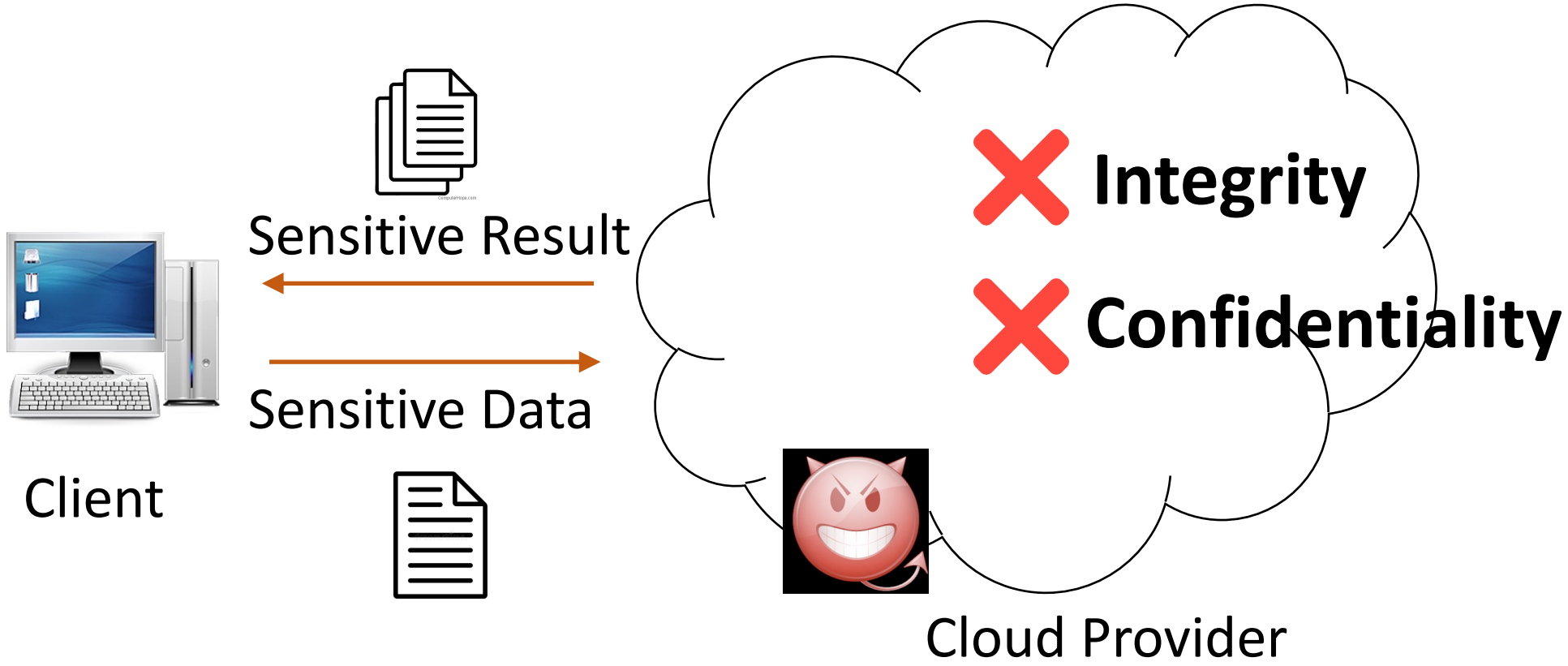
清華大學 交叉信息研究院
Institute for Interdisciplinary Information Sciences, Tsinghua University

上海期智研究院
SHANGHAI QI ZHI INSTITUTE

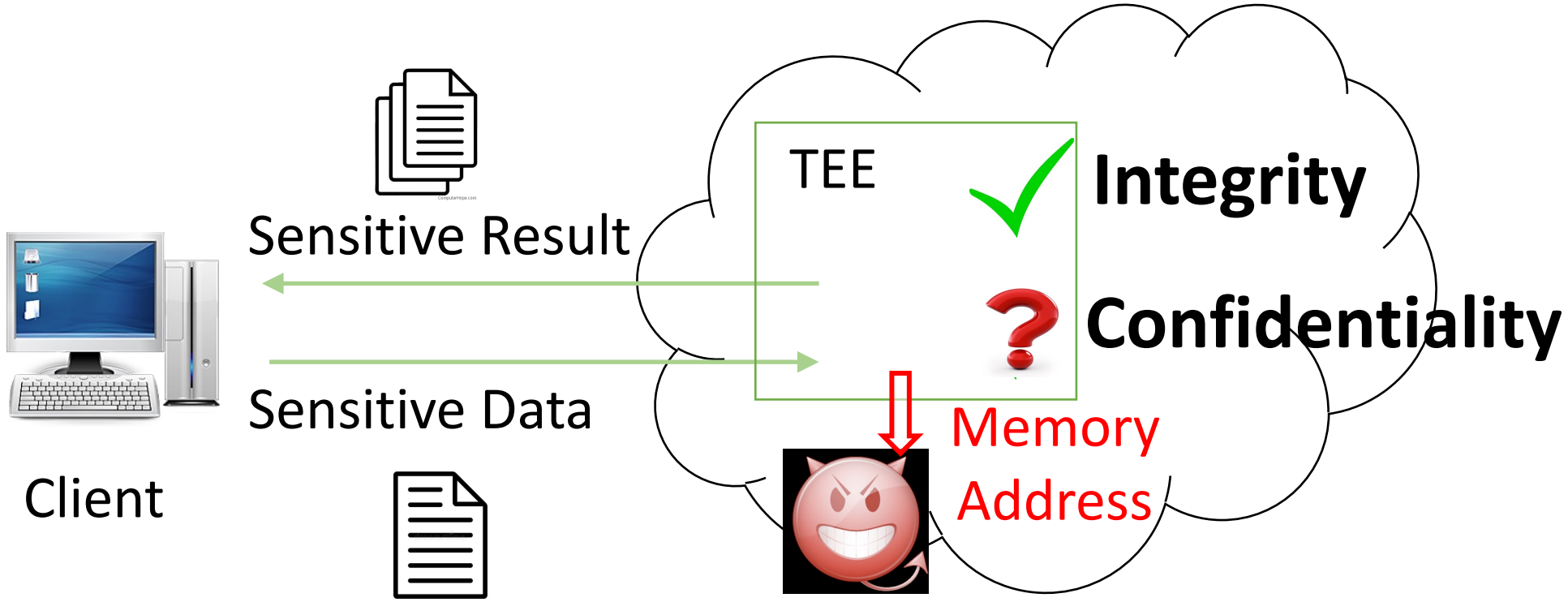
清華大學
Tsinghua University

IEEE S&P – May 2024

Cloud Computing on Sensitive Data



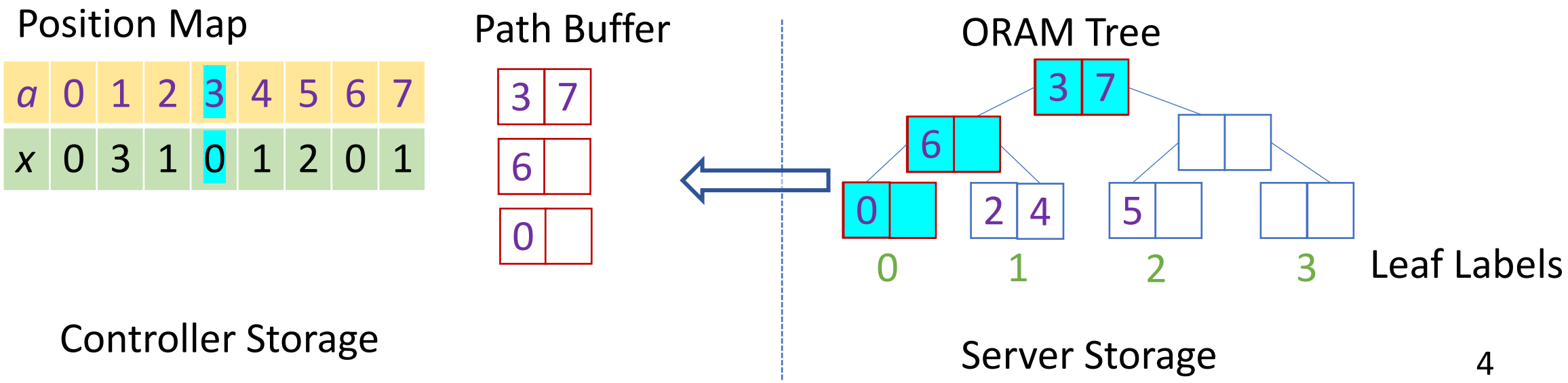
Trusted Execution Environment (TEE)



- E.g., Intel SGX, AMD SEV
- Problem: side channels, especially access-pattern-based side channels
  - Branch shadowing attacks [Lee et al., USENIX Sec'17]
  - Cache timing attacks [Brasser et al., WOOT'17][Ahmad Moghimi et al., CHES'17]
  - Page fault attacks [Xu et al., SP'15]
  - …

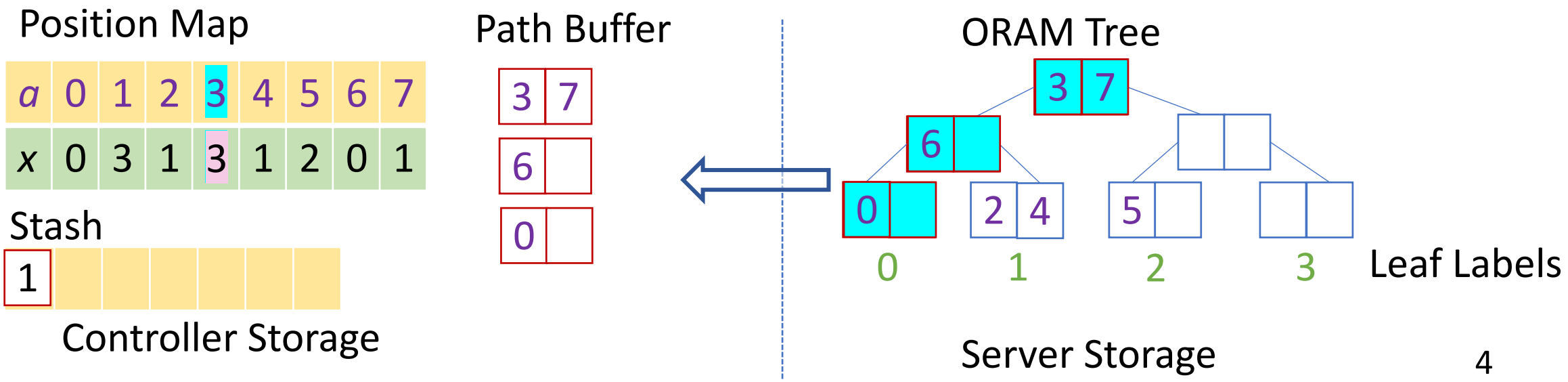
Path ORAM [Stefanov et al., 2013]

- Cryptographic primitive -> hide memory access pattern
- For each block access, a random path is fetched
- Main invariant: a block must reside on its path or stash



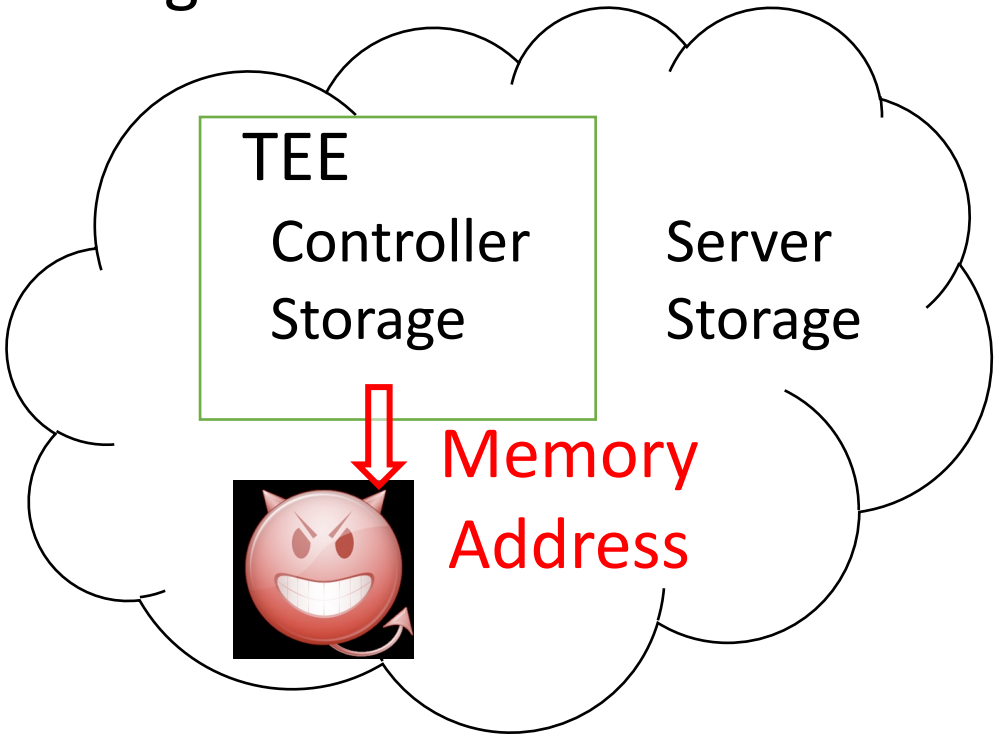
Path ORAM [Stefanov et al., 2013]

- Cryptographic primitive -> hide memory access pattern
- For each block access, a random path is fetched
- Main invariant: a block must reside on its path or stash



When Putting Path ORAM into TEE

- Move controller storage into TEE -> client access leakage



- Doubly-oblivious requirement ([Oblix, SP'18], [ZeroTrace, NDSS'18])
  - Accesses to the ORAM server are oblivious
  - Accesses to the ORAM (client) controller structures are oblivious
    - Codes inside TEE should execute obliviously!

ORAM Initialization w/ TEE?

- ORAM initialization w/ TEE is less studied
  - Naïve method: insert n blocks following the Path ORAM protocol
  - State-of-the-art: $O(n \log^3 n)$ in Oblix
- Building block for oblivious algorithms
  - Before accessing ORAM, initialization is needed
  - For e2e perf, initialization is one of the bottlenecks
    - Oblivious BFS
    - Oblivious query
- Data recovery
  - A long period of system unavailability
- Cloud storage services
  - ORAM structure has larger size (e.g., 4x)
  - Charge by capacity

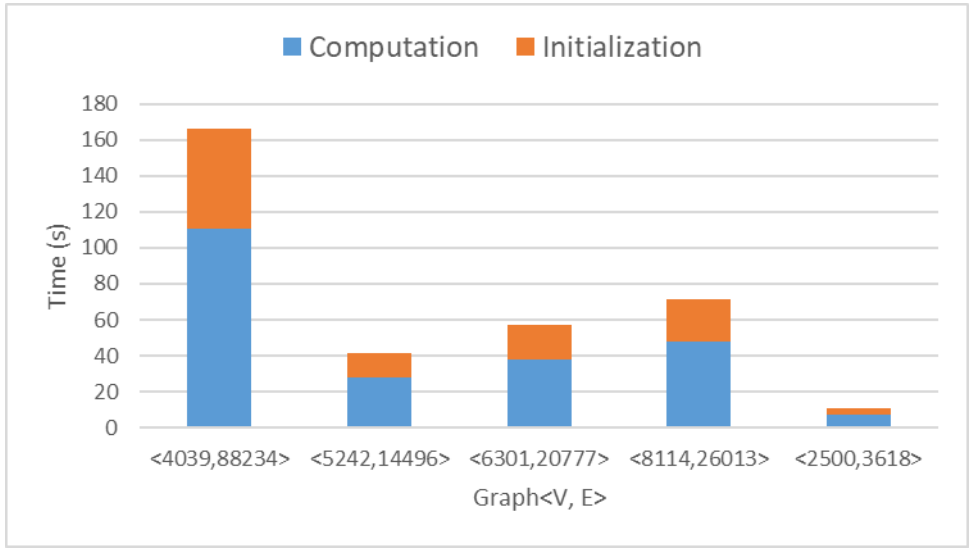


Fig 1. E2e perf of oblivious BFS

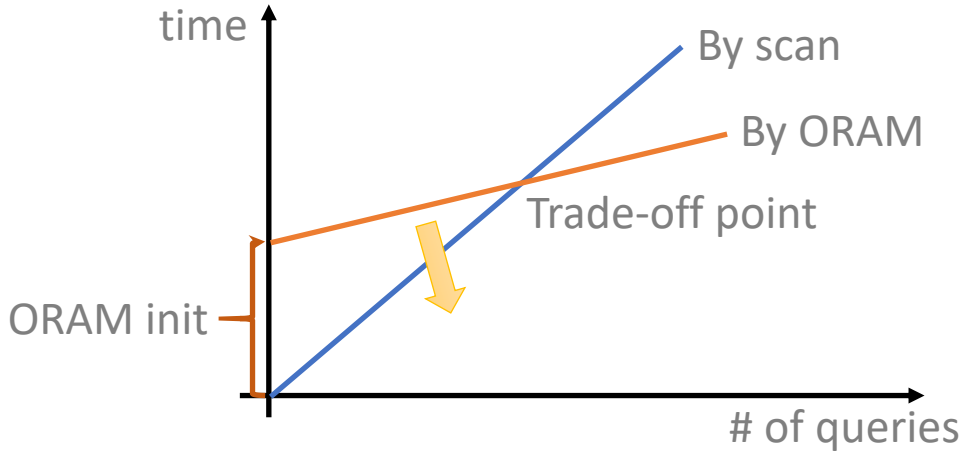
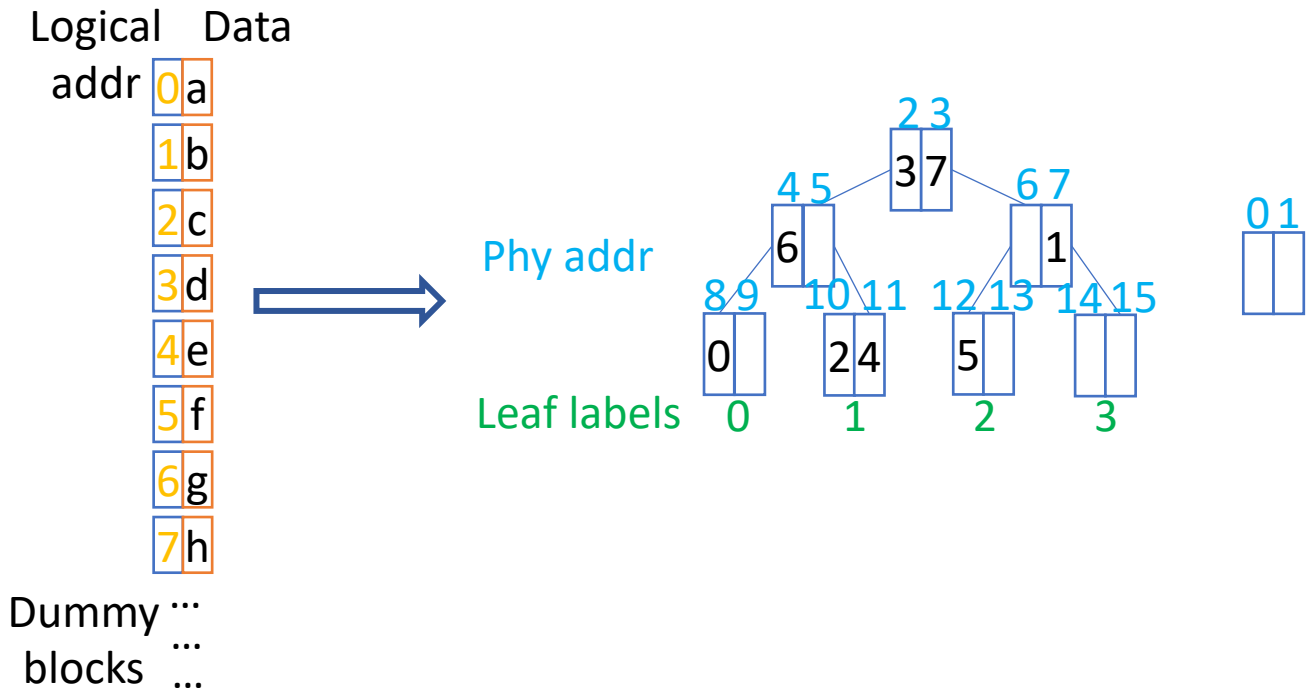


Fig 2. Two methods for oblivious query

ORAM Bulk Loading

- Problem definition
  - Given an array of blocks, each with logical addr and data, build a whole ORAM structure, including stash, position map, and ORAM tree.



ORAM Bulk Loading

- Problem definition
  - Given an array of blocks, each with logical addr and data, build a whole ORAM structure, including stash, position map, and ORAM tree.
- Goal
  - Performance
    - Theoretical complexity: $O(nlog^3 n) \rightarrow O(nlog^2 n)$
    - Practical high perf: e.g., parallelism
  - Security
    - Doubly oblivious
    - Leaf label distribution is consistent with Path ORAM
- Threat model
  - A malicious host controls the whole system
  - Hardware processor is trusted
  - Access pattern side channels

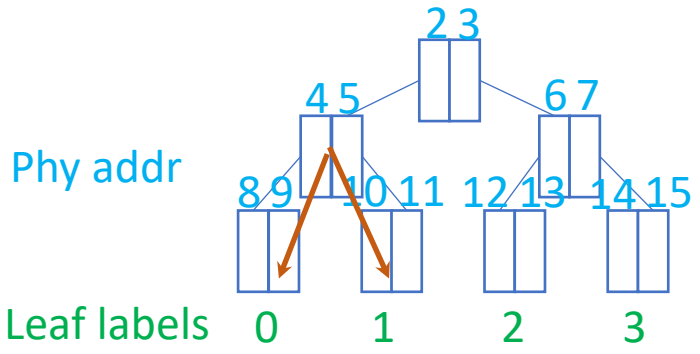
Simple but Insecure Constructions

- Random assignment for phy addr
  - Address collision
- Random block permutation

  -> determine phy addr

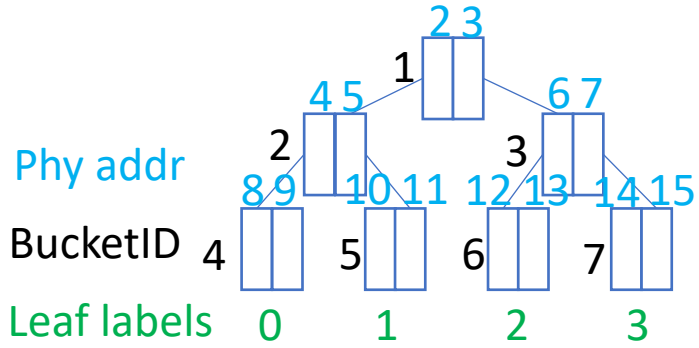
  -> assign leaf label (not uniformly distributed)

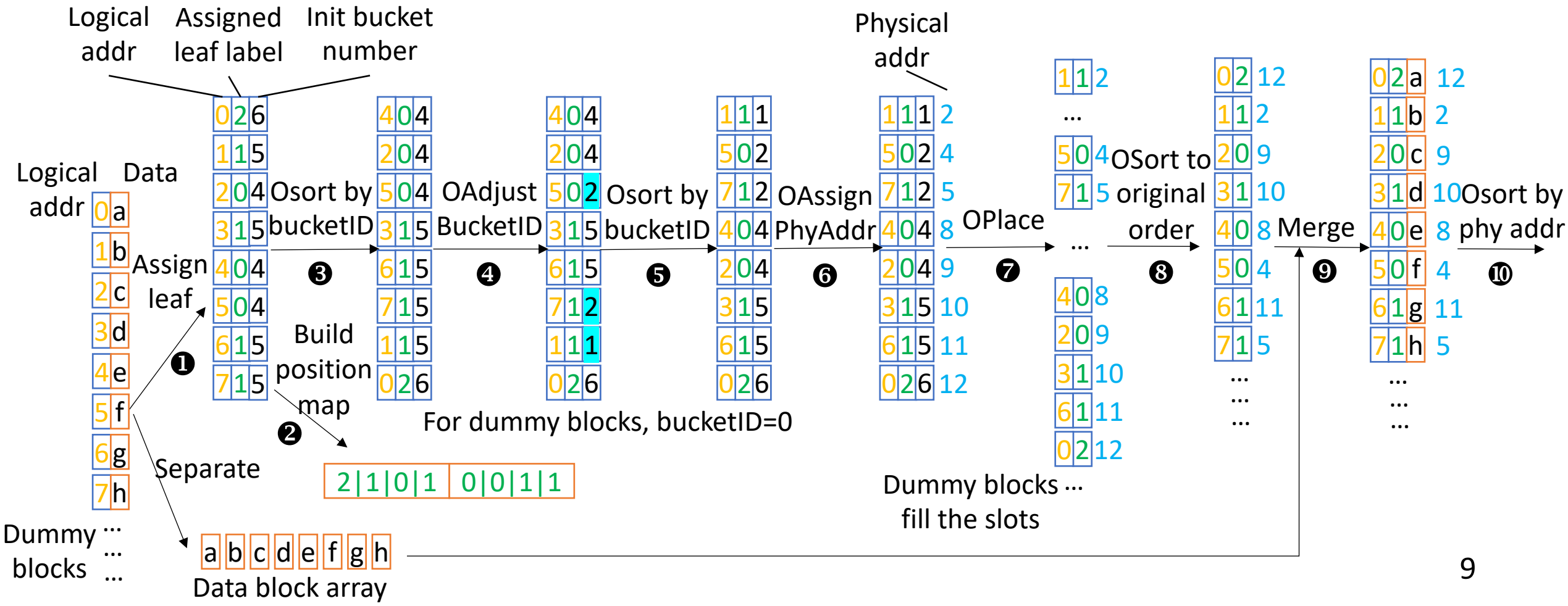
- We formally prove its insecurity



Bulkor

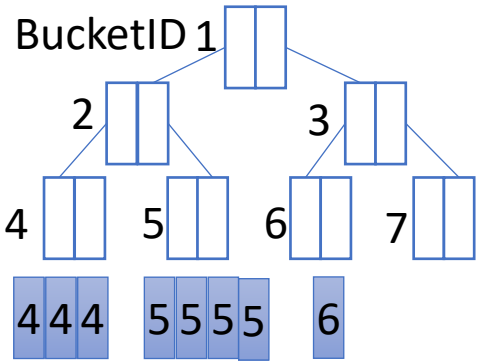
- Consistent leaf label distribution
- Bucket overflow



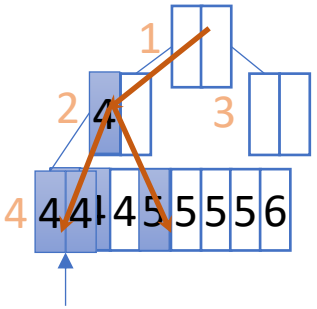


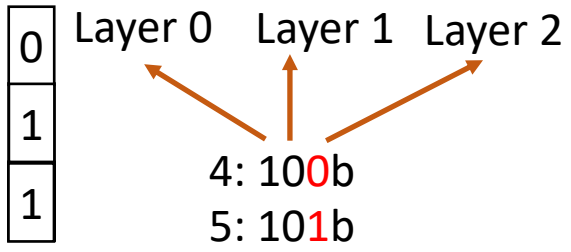
Oblivious Bucket ID Adjustment

- Main invariant of Path ORAM: each block stays on its path or stash
- Iteratively adjust the bucket ID to the parent until we find an empty slot
  - Obliviousness: occupancy counter update by scan
  - Efficiency: counter sharing: $O(n) \rightarrow O(\log n)$



Occupancy of the bucket on the current path





Counters below layer 2 (included) are cleared

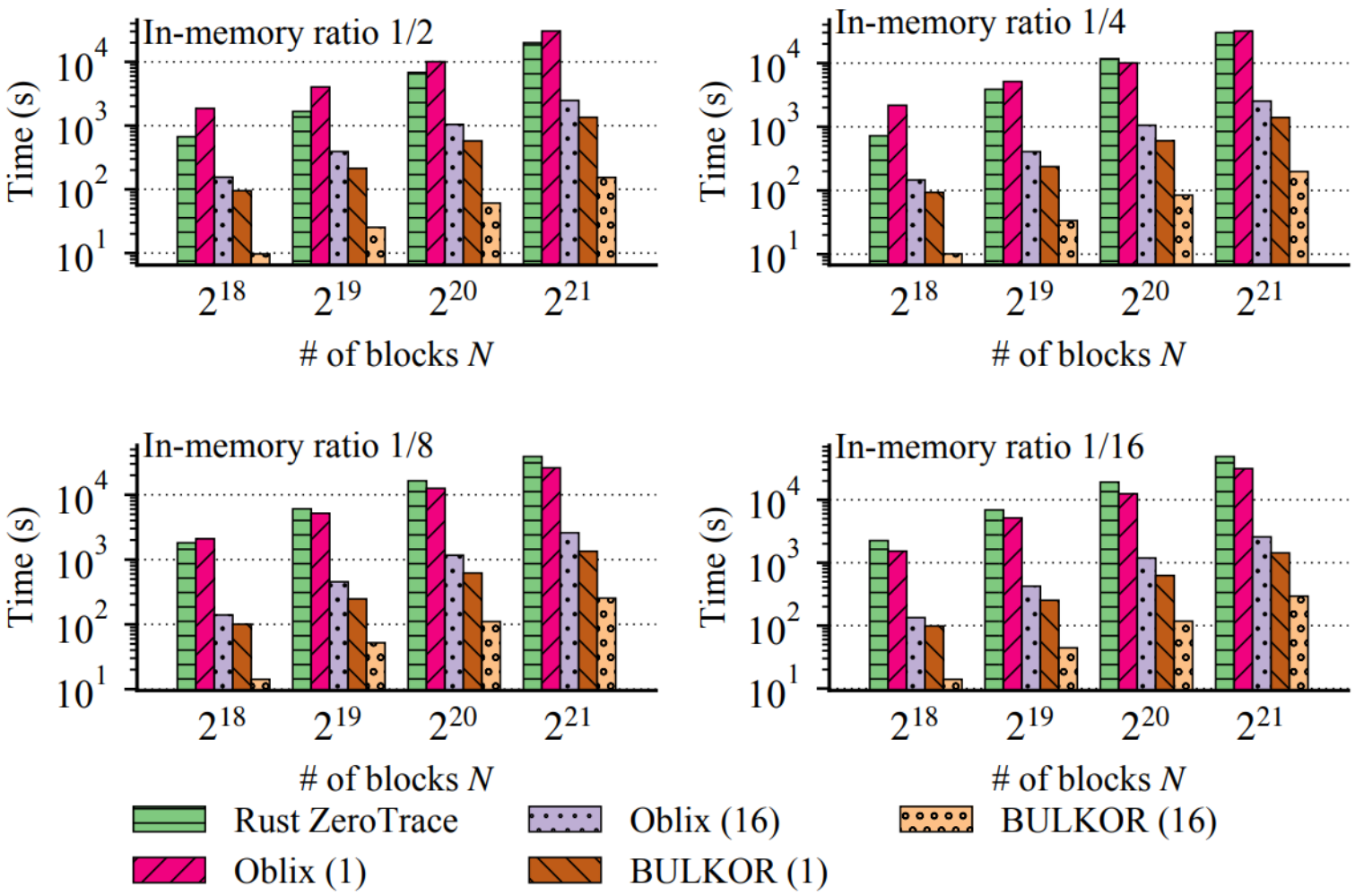
Only a single scan is needed to adjust the bucketID.
Time complexity: $O(nlogn)$.
The whole procedure is oblivious.

Evaluation

- Theoretical complexity: $O(n \log^3 n) \rightarrow O(n \log^2 n)$, bound on Osort
- Platform
  - Intel Xeon Gold 5317 processor w/ 64GB enclave page cache (EPC)
  - A single disk
- Parameters
  - Adjust block size & in-memory ratio & in-enclave ratio
  - Properly set to ensure comparison fairness
- Baselines
  - ZeroTrace [NDSS'18] (C++): naïve initialization
  - ZeroTrace (Rust): a re-implementation for fair comparison
  - Oblix [SP'18]: optimized initialization
  - Bulkor

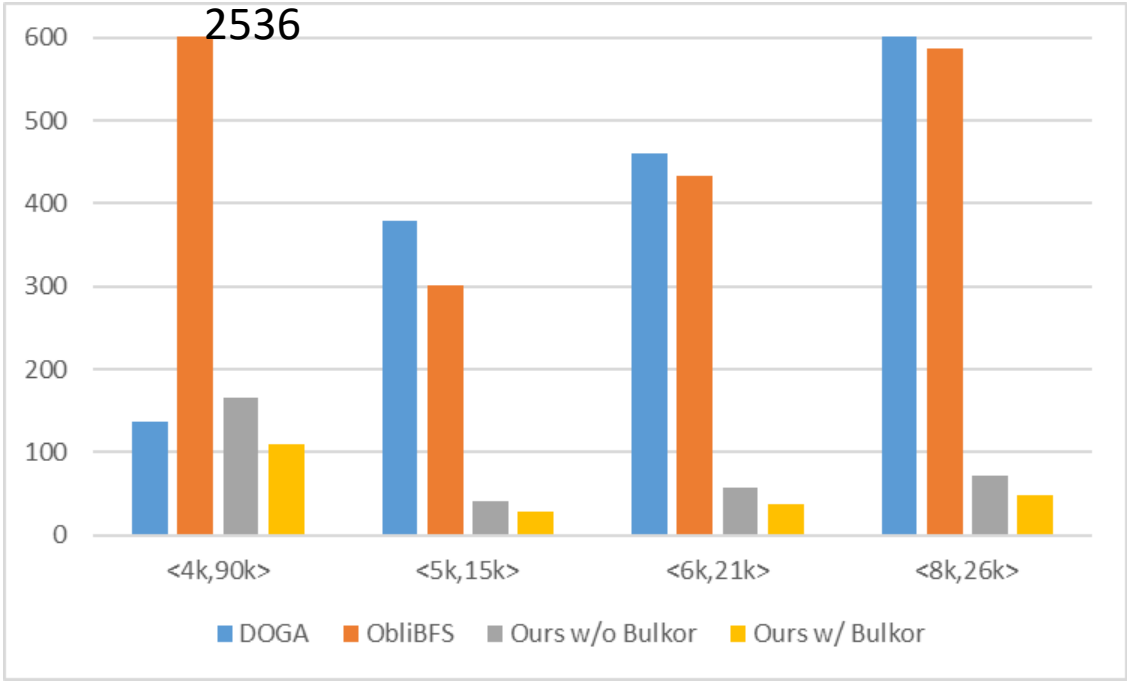
Overall performance

- Disk, 1kB block
- Cache treetop in memory
- 1-thread
  - 22.8x-34.2x speedup over Rust ZeroTrace
  - 15.5x-21.9x speedup over Oblix
- 16-thread
  - 139.1x-160.6x speedup over Rust Zerotrace
  - 8.7x-10.1x speedup over Oblix



Practical Application Case Studies

- Oblivious BFS with graph (V, E)
  - DOGA (Blanton et al., 2013).: custom alg not using ORAM
  - ObliBFS (Moreno-Sanchez et al., 2015): custom alg using ORAM
  - Ours: custom alg using ORAM
- Without BULKOR, our algorithm is slower than DOGA when the graph is dense (e.g., 4k vertices and 90k edges); BULKOR eliminates this degradation.



Oblivious BFS performance (in seconds)

More case studies in the paper!

Conclusion

- Formal definition of bulk loading program
- Efficient algorithm with better time complexity and practical perf
- Thorough system implementation
- Case studies of using Bulkor

Thank you!